Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC2223 - Teoría de Autómatas y Lenguajes Formales
Profesor: Cristian Riveros
Ayudante: Santiago Marcano

Ayudantía 8

On-The-Fly y Trasductores

Ejercicio 1

Considere el siguiente problema:

> **Problema:** Longest-Substring-NFA
> **Input:** Un NFA $A = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \cdots a_n \in \Sigma^*$.
> **Output:** Un par de índices (i, j), con $1 \leq i \leq j \leq n$, tal que $a_i \cdots a_j \in L(A)$ y $j - i + 1$ sea máximo entre todas las subpalabras de w aceptadas por A.

El problema Longest-Substring-NFA consiste en encontrar una subpalabra contigua de w de largo máximo que sea aceptada por el NFA A. Si ninguna subpalabra no vacía de w pertenece a $L(A)$, retornar none.

Escriba un algoritmo *on-the-fly* que resuelva Longest-Substring-NFA en tiempo $O(|A| \cdot |w|)$. Demuestre la correctitud de su algoritmo utilizando un invariante y analice su complejidad.

Solución Ejercicio 1

La idea es modificar el algoritmo *on-the-fly*. En vez de mantener solamente el conjunto de estados alcanzables, para cada estado alcanzable q almacenaremos además la posición más a la izquierda desde la cual podemos llegar a q mediante una subpalabra que termina en la posición actual.

En particular, después de procesar la posición i, si

$$S[q] = j,$$

entonces j será el menor índice tal que existe una ejecución de A sobre

$$a_j \cdots a_i$$

que termina en q.

Algoritmo 1 LONGEST-SUBSTRING-NFA(A, w)

```
1:  S ← ∅
2:  best ← NONE
3:  bestLength ← 0
4:  for i = 1 to n do
5:      S_old ← S
6:      S ← ∅
7:      Comenzar nuevas subpalabras en la posición i
8:      for all p ∈ I do
9:          for all q tal que (p, a_i, q) ∈ Δ do
10:             S[q] ← i
11:         end for
12:     end for
13:     Extender las subpalabras que ya estaban activas
14:     for all p ∈ S_old do
15:         for all q tal que (p, a_i, q) ∈ Δ do
16:             if q ∉ S then
17:                 S[q] ← S_old[p]
18:             else
19:                 S[q] ← min{S[q], S_old[p]}
20:             end if
21:         end for
22:     end for
23:     Revisar las subpalabras aceptadas que terminan en i
24:     for all q ∈ F tal que q ∈ S do
25:         j ← S[q]
26:         if i − j + 1 > bestLength then
27:             best ← (j, i)
28:             bestLength ← i − j + 1
29:         end if
30:     end for
31: end for
32: return best
```

Para demostrar la correctitud consideramos el siguiente invariante.

Invariante. Después de procesar las primeras i posiciones de la palabra, para cada estado q presente en S, $S[q]$ es el menor índice j tal que existe una ejecución de A sobre

$$a_j \cdots a_i$$

que termina en q.

De esto mismo se concluye que, si q no aparece en S, entonces no existe ninguna subpalabra no vacía que termine en la posición i y cuya ejecución termine en q.

Demostraremos el invariante por inducción sobre i.

Caso base ($i = 0$). Antes de comenzar el ciclo tenemos

$$S = \emptyset.$$

Como todavía no existe ninguna posición procesada, tampoco existe una subpalabra no vacía que termine en la posición 0. Por lo tanto, ningún estado debe estar presente en S y el invariante se cumple.

Paso inductivo. Supongamos que el invariante se cumple después de procesar la posición $i-1$. Al comenzar la siguiente iteración, el algoritmo realiza

$$S_{\text{old}} \leftarrow S \quad \text{y} \quad S \leftarrow \emptyset.$$

Por hipótesis inductiva, para cada $p \in S_{\text{old}}$, $S_{\text{old}}[p]$ es el menor índice desde el cual existe una subpalabra que termina en $i-1$ y cuya ejecución termina en p.

Consideremos ahora las subpalabras que terminan en la posición i para las cuales existen dos posibilidades:

Primero, una subpalabra puede comenzar exactamente en la posición i, en este caso contiene únicamente la letra a_i. Si existe $p \in I$ y una transición

$$(p, a_i, q) \in \Delta,$$

entonces existe una ejecución sobre a_i que termina en q, por lo que el algoritmo agrega

$$S[q] = i.$$

Segundo, una subpalabra puede haber comenzado antes de la posición i. Supongamos que existe una ejecución que, después de leer hasta la posición $i-1$, se encuentra en p, y que

$$(p, a_i, q) \in \Delta.$$

Entonces podemos extender dicha ejecución leyendo a_i y llegar a q.

Por hipótesis inductiva, el menor índice desde el cual podemos llegar a p es

$$S_{\text{old}}[p].$$

Por lo tanto, este mismo índice es candidato para el valor de $S[q]$.

Si distintas ejecuciones llegan al mismo estado q, el algoritmo conserva el menor de sus índices iniciales mediante

$$S[q] \leftarrow \min(S[q], S_{\text{old}}[p]).$$

Así, después de procesar la posición i, $S[q]$ contiene exactamente el menor índice desde el cual existe una ejecución que termina en q. Concluimos entonces que el invariante se mantiene.

Correctitud del algoritmo

Después de procesar la posición i, supongamos que $q \in F$ y $q \in S$. Por el invariante, existe una ejecución sobre

$$a_{S[q]} \cdots a_i$$

que termina en el estado final q. Por lo tanto,

$$a_{S[q]} \cdots a_i \in L(A).$$

Además, $S[q]$ es el menor índice desde el cual es posible llegar a q. Por lo tanto,

$$a_{S[q]} \cdots a_i$$

es la subpalabra más larga que termina en i y cuya ejecución termina en q.

Supongamos que

$$a_j \cdots a_i$$

es una subpalabra aceptada. Entonces existe una ejecución sobre dicha subpalabra que termina en algún estado final $q \in F$.

Por el invariante,

$$S[q] \leq j.$$

Por lo tanto,

$$i - S[q] + 1 \geq i - j + 1.$$

Es decir, cuando el algoritmo revisa el estado final q, considera una subpalabra aceptada que es al menos tan larga como $a_j \cdots a_i$.

Como el algoritmo realiza esta revisión para todos los estados finales y para todas las posiciones i, ninguna subpalabra aceptada puede ser más larga que la almacenada finalmente en *best*.

Por lo tanto, *best* contiene una subpalabra aceptada de largo máximo.

Complejidad

En cada posición se recorren los estados activos y las transiciones relevantes del NFA. En el peor caso esto requiere tiempo $O(|A|)$.

Como existen $|w|$ posiciones, el tiempo total es

$$O(|A| \cdot |w|).$$

Los mapas S y S_{old} contienen a lo más una entrada por estado, por lo que utilizan espacio

$$O(|Q|).$$

Ejercicio 2

Considere el siguiente problema:

> **Problema:** T-Eval
> **Input:** Un transductor no determinista $T = (Q, \Sigma, \Omega, \Delta, I, F)$ y una palabra $u = a_1 \cdots a_n \in \Sigma^*$.
> **Output:** El conjunto $V \subseteq \Omega^*$ tal que $V = [\![T]\!](u)$

Puede asumir que T no posee transiciones que lean ε.

Diseñe un algoritmo que resuelva T-Eval retornando el conjunto de las traducciones hechas por el transductor dado el input dado, su algoritmo debe tener complejidad $\mathcal{O}(|w| \cdot |Q|)$ Demuestre la correctitud de su algoritmo.

Solución Ejercicio 2

Adaptaremos nuevamente el algoritmo *on-the-fly*.

En este caso, no basta con saber qué estados del transductor son alcanzables. También debemos recordar qué palabra de salida fue producida por cada ejecución.

Por lo tanto, S será un conjunto de pares

$$(q, v) \in Q \times \Omega^*,$$

donde $(q, v) \in S$ significa que, después de leer el prefijo procesado, existe una ejecución del transductor que se encuentra en el estado q y que ha producido la palabra v.

Como el transductor no posee transiciones que lean ε, cada iteración del algoritmo consume exactamente una letra de la entrada.

Algoritmo 2 T-EVAL(T, u)

```
1: S ← {(q, ε) | q ∈ I}
2: for i = 1 to n do
3:     S_old ← S
4:     S ← ∅
5:     for all (p, v) ∈ S_old do
6:         for all (p, a_i, x, q) ∈ Δ do
7:             S ← S ∪ {(q, v · x)}
8:         end for
9:     end for
10: end for
11: V ← {v | (q, v) ∈ S para algún q ∈ F}
12: return V
```

Aquí $x \in \Omega^*$ representa la palabra producida por la transición. En particular, x puede ser una letra, una palabra o ε, dependiendo del transductor.

Para demostrar la correctitud consideramos el siguiente invariante.

Invariante. Después de procesar las primeras i letras de la entrada, se cumple que

$$(q, v) \in S$$

si, y solo si, existe una ejecución de T sobre el prefijo

$$a_1 \cdots a_i$$

que comienza en algún estado inicial, termina en q y produce exactamente la palabra v.

Demostraremos el invariante por inducción sobre i.

Caso base ($i = 0$). Antes de leer cualquier símbolo, el algoritmo inicializa

$$S = \{(q, \varepsilon) \mid q \in I\}.$$

Sin leer ninguna letra, las únicas ejecuciones posibles son las ejecuciones de largo cero que comienzan en algún estado inicial $q \in I$. Estas ejecuciones producen la palabra vacía ε.

Paso inductivo. Supongamos que el invariante se cumple después de procesar $a_1 \cdots a_{i-1}$.

El algoritmo guarda

$$S_{\text{old}} \leftarrow S \quad \text{y} \quad S \leftarrow \emptyset.$$

Debemos demostrar que el nuevo S contiene exactamente los pares correspondientes a las ejecuciones sobre $a_1 \cdots a_i$.

Primero, sea

$$(q, v') \in S.$$

Por construcción del algoritmo, existen un par

$$(p, v) \in S_{\text{old}}$$

y una transición

$$(p, a_i, x, q) \in \Delta$$

tales que

$$v' = v \cdot x.$$

Por hipótesis inductiva, existe una ejecución sobre $a_1 \cdots a_{i-1}$ que termina en p y produce v. Agregando la transición (p, a_i, x, q) obtenemos una ejecución sobre $a_1 \cdots a_i$ que termina en q y produce

$$v \cdot x = v'.$$

Por lo tanto, todo par agregado a S corresponde a una ejecución real del transductor.

Correctitud del algoritmo

Después de terminar el ciclo se han procesado las n letras de la palabra u.

Por el invariante,

$$(q, v) \in S$$

si, y solo si, existe una ejecución completa de T sobre u que termina en q y produce v.

Una ejecución del transductor es de aceptación precisamente cuando su estado final pertenece a F. Por lo tanto,

$$v \in [\![T]\!](u)$$

si, y solo si, existe $q \in F$ tal que

$$(q, v) \in S.$$

El algoritmo retorna precisamente

$$V = \{v \mid (q, v) \in S \text{ para algún } q \in F\}.$$

En consecuencia,

$$V = [\![T]\!](u),$$

y el algoritmo es correcto.

Complejidad

A diferencia de la evaluación de un NFA, el tamaño de S no está necesariamente acotado por $|Q|$. Un mismo estado puede ser alcanzado mediante muchas ejecuciones que producen palabras de salida distintas.

Por lo tanto, el número de pares almacenados en S puede crecer exponencialmente con el largo de la entrada.

Esto es inevitable en general, ya que el conjunto $[\![T]\!](u)$ que debemos retornar puede contener una cantidad exponencial de palabras.

Ejercicio 3

Decimos que un transductor es sincrono si $\Delta \subseteq Q \times \Sigma \times \Gamma \times Q$. Sean $T_1 = (Q_1, \Sigma, \Gamma, \Delta_1, I_1, F_1)$ y $T_2 = (Q_2, \Gamma, \Omega, \Delta_2, I_2, F_2)$ dos transductores síncronos.

Definimos la composición de las relaciones definidas por T_1 y T_2 como

$$[\![T_2]\!] \circ [\![T_1]\!] = \{(u, w) \in \Sigma^* \times \Omega^* \mid \exists v \in \Gamma^* : (u, v) \in [\![T_1]\!] \wedge (v, w) \in [\![T_2]\!]\}.$$

Esto es, una palabra u es transformada primero por T_1 en una palabra intermedia v, y luego T_2 transforma v en una palabra w.

Demuestre que existe un transductor síncrono T tal que

$$[\![T]\!] = [\![T_2]\!] \circ [\![T_1]\!].$$

Solución Ejercicio 3

Construiremos un transductor que simula simultáneamente a T_1 y T_2.

Recordemos que $T_1 = (Q_1, \Sigma, \Gamma, \Delta_1, I_1, F_1)$ transforma palabras sobre Σ en palabras sobre Γ, mientras que $T_2 = (Q_2, \Gamma, \Omega, \Delta_2, I_2, F_2)$ transforma palabras sobre Γ en palabras sobre Ω.

Como ambos transductores son síncronos, una transición de T_1 tiene la forma

$$p_1 \xrightarrow{a/b} q_1,$$

mientras que una transición de T_2 tiene la forma

$$p_2 \xrightarrow{b/c} q_2.$$

Si la letra producida por T_1 coincide con la letra consumida por T_2, podemos combinar ambas transiciones en

$$(p_1, p_2) \xrightarrow{a/c} (q_1, q_2).$$

Definimos

$$T = (Q, \Sigma, \Omega, \Delta, I, F)$$

de la siguiente manera:

$$Q = Q_1 \times Q_2,$$
$$I = I_1 \times I_2,$$
$$F = F_1 \times F_2.$$

Definimos la relación de transición como

$$\Delta = \big\{((p_1, p_2), a, c, (q_1, q_2)) \;\big|\; \exists b \in \Gamma \text{ tal que } (p_1, a, b, q_1) \in \Delta_1 \text{ y } (p_2, b, c, q_2) \in \Delta_2\big\}.$$

Como cada transición de T consume exactamente una letra de Σ y produce exactamente una letra de Ω, T es también un transductor síncrono.

Demostraremos que

$$[\![T]\!] = [\![T_2]\!] \circ [\![T_1]\!]$$

mediante doble contención.

$[\![T]\!] \subseteq [\![T_2]\!] \circ [\![T_1]\!]$

Sea

$$(u, w) \in [\![T]\!].$$

Como T es síncrono, podemos escribir

$$u = a_1 \cdots a_n \qquad \text{y} \qquad w = c_1 \cdots c_n.$$

Como $(u, w) \in [\![T]\!]$, existe una ejecución de aceptación de T de la forma

$$(p_0, r_0) \xrightarrow{a_1/c_1} (p_1, r_1) \xrightarrow{a_2/c_2} \cdots \xrightarrow{a_n/c_n} (p_n, r_n),$$

donde $(p_0, r_0) \in I_1 \times I_2$ y $(p_n, r_n) \in F_1 \times F_2$.

Por definición de Δ, para cada $i \in \{1, \ldots, n\}$ existe una letra $b_i \in \Gamma$ tal que

$$(p_{i-1}, a_i, b_i, p_i) \in \Delta_1 \quad \text{y} \quad (r_{i-1}, b_i, c_i, r_i) \in \Delta_2.$$

Definamos $v = b_1 \cdots b_n$. Las primeras transiciones forman una ejecución de T_1:

$$p_0 \xrightarrow{a_1/b_1} p_1 \xrightarrow{a_2/b_2} \cdots \xrightarrow{a_n/b_n} p_n.$$

Como

$$p_0 \in I_1 \qquad \text{y} \qquad p_n \in F_1,$$

esta ejecución es de aceptación. Por lo tanto,

$$(u, v) \in [\![T_1]\!].$$

De manera análoga, las segundas transiciones forman una ejecución de aceptación de T_2:

$$r_0 \xrightarrow{b_1/c_1} r_1 \xrightarrow{b_2/c_2} \cdots \xrightarrow{b_n/c_n} r_n.$$

Así,

$$(v, w) \in [\![T_2]\!].$$

Por lo tanto, existe $v \in \Gamma^*$ tal que

$$(u, v) \in [\![T_1]\!] \qquad \text{y} \qquad (v, w) \in [\![T_2]\!].$$

Por definición de composición,

$$(u, w) \in [\![T_2]\!] \circ [\![T_1]\!].$$

Concluimos que

$$[\![T]\!] \subseteq [\![T_2]\!] \circ [\![T_1]\!].$$

$[\![T_2]\!] \circ [\![T_1]\!] \subseteq [\![T]\!]$

Sea$(u, w) \in [\![T_2]\!] \circ [\![T_1]\!]$ por definición de composición, existe una palabra $v \in \Gamma^*$ tal que

$$(u, v) \in [\![T_1]\!] \quad \text{y} \quad (v, w) \in [\![T_2]\!].$$

Como ambos transductores son síncronos,

$$|u| = |v| = |w|.$$

Escribamos

$$u = a_1 \cdots a_n, \qquad v = b_1 \cdots b_n, \qquad w = c_1 \cdots c_n.$$

Como

$$(u, v) \in [\![T_1]\!],$$

existe una ejecución de aceptación

$$p_0 \xrightarrow{a_1/b_1} p_1 \xrightarrow{a_2/b_2} \cdots \xrightarrow{a_n/b_n} p_n,$$

donde

$$p_0 \in I_1 \qquad \text{y} \qquad p_n \in F_1.$$

Además, como

$$(v, w) \in [\![T_2]\!],$$

existe una ejecución de aceptación

$$r_0 \xrightarrow{b_1/c_1} r_1 \xrightarrow{b_2/c_2} \cdots \xrightarrow{b_n/c_n} r_n,$$

donde

$$r_0 \in I_2 \qquad \text{y} \qquad r_n \in F_2.$$

Para cada posición i tenemos

$$(p_{i-1}, a_i, b_i, p_i) \in \Delta_1 \quad \text{y} \quad (r_{i-1}, b_i, c_i, r_i) \in \Delta_2.$$

Por definición de Δ, esto implica que

$$((p_{i-1}, r_{i-1}), a_i, c_i, (p_i, r_i)) \in \Delta.$$

Por lo tanto, T posee la ejecución

$$(p_0, r_0) \xrightarrow{a_1/c_1} (p_1, r_1) \xrightarrow{a_2/c_2} \cdots \xrightarrow{a_n/c_n} (p_n, r_n).$$

Donde

$$(p_0, r_0) \in I_1 \times I_2 = I \quad \text{y} \quad (p_n, r_n) \in F_1 \times F_2 = F.$$

Por lo tanto, esta es una ejecución de aceptación de T, y concluimos que

$$(u, w) \in [\![T]\!].$$

Así,

$$[\![T_2]\!] \circ [\![T_1]\!] \subseteq [\![T]\!].$$

Por ambas inclusiones,

$$\boxed{[\![T]\!] = [\![T_2]\!] \circ [\![T_1]\!]}.$$